Reviewer Pack — Minimal Hodge Diamond Diameter and Communication Complexity …

Entry 2f35b4372a33 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-07 21:08:34 UTC

Minimal Hodge Diamond Diameter and Communication Complexity Rank

Entry ID: `2f35b4372a33` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-07 21:08:34 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Hodge Theory) **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

> For any Boolean function f with m inputs, the diameter of the minimal Hodge diamond of its algebraic variety X_f is linearly correlated with its communication complexity rank r(f), such that d(X_f) = O(r(f))

Rationale (proposer's reasoning):

> Hodge theory provides a way to study geometric properties of algebraic varieties. If the diameter of the Hodge diamond, which measures a certain geometric complexity, is closely related to the communication complexity rank, it suggests a deep connection between geometric structures and computational complexity.

Taxonomy category: `HODGE_DIAINDEX_RANK` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `7e39bbd298b8a163`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if the correlation coefficient between the diameter of the minimal Hodge diamond d(X_f) and communication complexity rank r(f) for all Boolean functions f is greater than or equal to 0.8 over 30 random seeds, with no seed producing a correlation coefficient less than 0.6.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.90	HITS	UNCERTAIN
KARP_LIPTON	SAFE	1.00	SAFE	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `'Hodge Theory' AND 'communication complexity rank' AND 'algebraic variety'` - `'matrix rank' IN 'Communication Complexity' AND 'diameter of Hodge diamond' AND 'Algebraic Geometry'` - `minimizing diameter of Hodge diamond IN 'Algebraic Geometry' AND related TO 'communication complexity rank'`

Top relevant hits considered: - [http://arxiv.org/abs/1102.2932v2] Applications of Monotone Rank to Complexity Theory - [http://arxiv.org/abs/2211.07055v3] Geometric complexity theory for product-plus-power - [http://arxiv.org/abs/2401.14623v1] Structure in Communication Complexity and Constant-Cost Complexity Classes - [http://arxiv.org/abs/1801.10489v3] Hodge level for weighted complete intersections - [http://arxiv.org/abs/2209.00110v4] Complexity of diameter on AT-free graphs is linear - [http://arxiv.org/abs/1607.01412v2] Pink's theory of Hodge structures and the Hodge conjecture over function fields

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=0, elapsed=0.0s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def construct_algebraic_variety(f):
        n = len(f)
        x = [0] * (n + 1)
        for i in range(n):
            if f[i] == '1':
                x[i] = 1
        return x

    def communication_complexity_rank(f):
        n = len(f)
        rank = 0
        for i in range(1, n):
            rank += math.log2(i + 1)
        return rank

    def hodge_diamond_diameter(poly):
        n = len(poly) - 1
        if n < 3:
            return 0
```

```
        d = [poly[i] * poly[n-i] for i in range(1, n//2+1)]
        return max(d)

    results = []
    for _ in range(30):
        n = random.choice([5, 10, 15, 20, 30, 40])
        f = ''.join(random.choices(['0', '1'], k=n))
        poly = construct_algebraic_variety(f)
        rank = communication_complexity_rank(f)
        diameter = hodge_diamond_diameter(poly)
        results.append((diameter, rank))

    correlation_coefficient = 0
    if len(results) > 1:
        x_mean = sum(d for d, r in results) / len(results)
        y_mean = sum(r for d, r in results) / len(results)
        numerator = sum((d - x_mean) * (r - y_mean) for d, r in results)
        denominator = math.sqrt(sum((d - x_mean)**2 for d, r in results)) * math.sqrt(sum((r - y_m
        correlation_coefficient = numerator / denominator if denominator != 0 else 0

    return {
        "metric_name": "correlation_coefficient",
        "metric_value": correlation_coefficient,
        "instances_tested": len(results),
        "n_max": max(n for _, _ in results),
        "conjecture_holds": correlation_coefficient >= 0.8,
        "counterexample": "" if correlation_coefficient >= 0.8 else "correlation_coefficient < 0.8
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43,
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif any(not r["conjecture_holds"] and r["counterexample"] == "correlation_coefficient < 0.8"
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"correlation_coefficient < 0.8\" first_failing_s
    else:
        print("RESULT: INCONCLUSIVE insufficient_data_or_other_reasons")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
ation_coefficient < 0.8'}
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': 0.4831021674280194, 'instances_te
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': 0.3932314107144544, 'instances_te
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': 0.3244428889711135, 'instances_te
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': 0.2192867825406661, 'instances_te
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': 0.2408990519583312, 'instances_te
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': 0.26730659085838737, 'instances_t
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': 0.4707565373363545, 'instances_te
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': 0.12124587979959729, 'instances_t
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': 0.3561795304215634, 'instances_te
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': 0.34181683800274887, 'instances_t
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> The test only considers a small range of n values (up to 40), which may not be sufficient to establish the conjecture's validity for all Boolean functions. The correlation coefficient is below the threshold, suggesting that the conjecture does not hold for at least some instances.

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> Safety rail: critic_challenge_falsified | original: The correlation coefficient between the diameter of the minimal Hodge diamond and communication complexity rank for at least one seed was below the th | next: Further investigation is needed to determine if there are specific types of Boolean functions that do not conform to the conjecture. Additionally, a broader range of n values should be tested to assess the conjecture's validity across different instances.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13154
2	propose	ollama_remote	glm4:latest	0	0	11908
3	preregistration	ollama_remote	glm4:latest	0	0	9268
4	novelty	ollama_remote	glm4:latest	0	0	8661
5	novelty	ollama_remote	glm4:latest	0	0	9458
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20625
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13487
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7054
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11266
10	critic	ollama_remote	glm4:latest	0	0	16351
11	judge	ollama_remote	glm4:latest	0	0	10922

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 132153 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in `research/audit/2f35b4372a33.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/2f35b4372a33.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/2f35b4372a33.tar.gz` (if generated)